https://www.kaggle.com/c/dogs-vs-cats/data

kaggle Search Competitions Datasets Kernels Discussion Learn ...

Dogs vs. Cats

Create an algorithm to distinguish dogs from cats

215 teams · 5 years ago

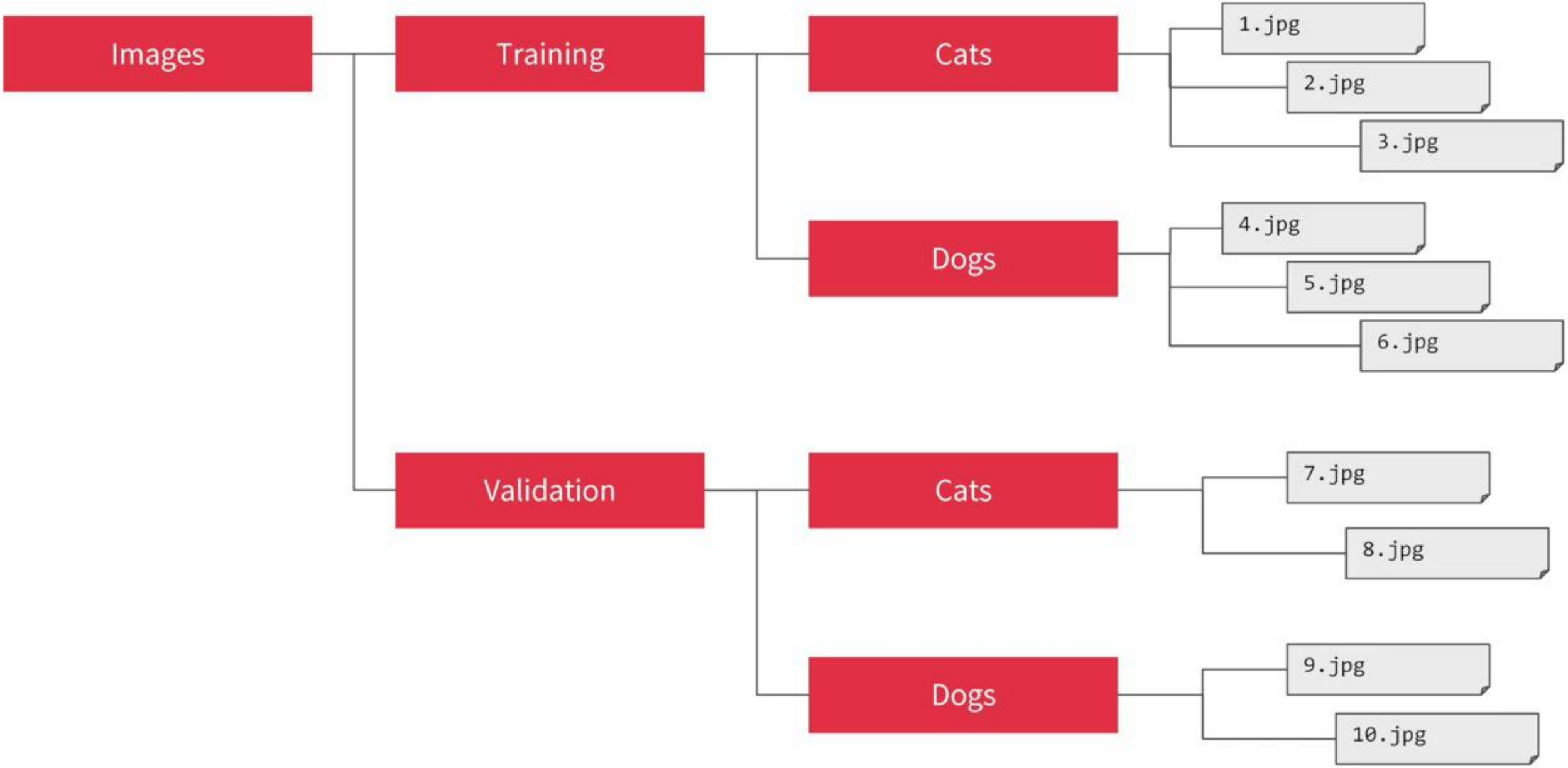
Overview Data Kernels Discussion Leaderboard Rules Team

Data Description

The training archive contains 25,000 images of dogs and cats. Train your algorithm on these files and predict the labels for test1.zip (1 = dog, 0 = cat).

A note on hand labeling

Per the rules and spirit of this contest, please do not manually label your submissions. We work hard to fair and fun contests, and ask for the same respect in return.


Images
Training
Cats
1.jpg
2.jpg
3.jpg
Dogs
4.jpg
5.jpg
6.jpg
Validation
Cats
7.jpg
8.jpg
Dogs
9.jpg
10.jpg

```
from tensorflow.keras.preprocessing.image
import ImageDataGenerator
```

```
train_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
        train_dir,
        target_size=(150, 150),
        batch_size=20,
        class_mode='binary')
```

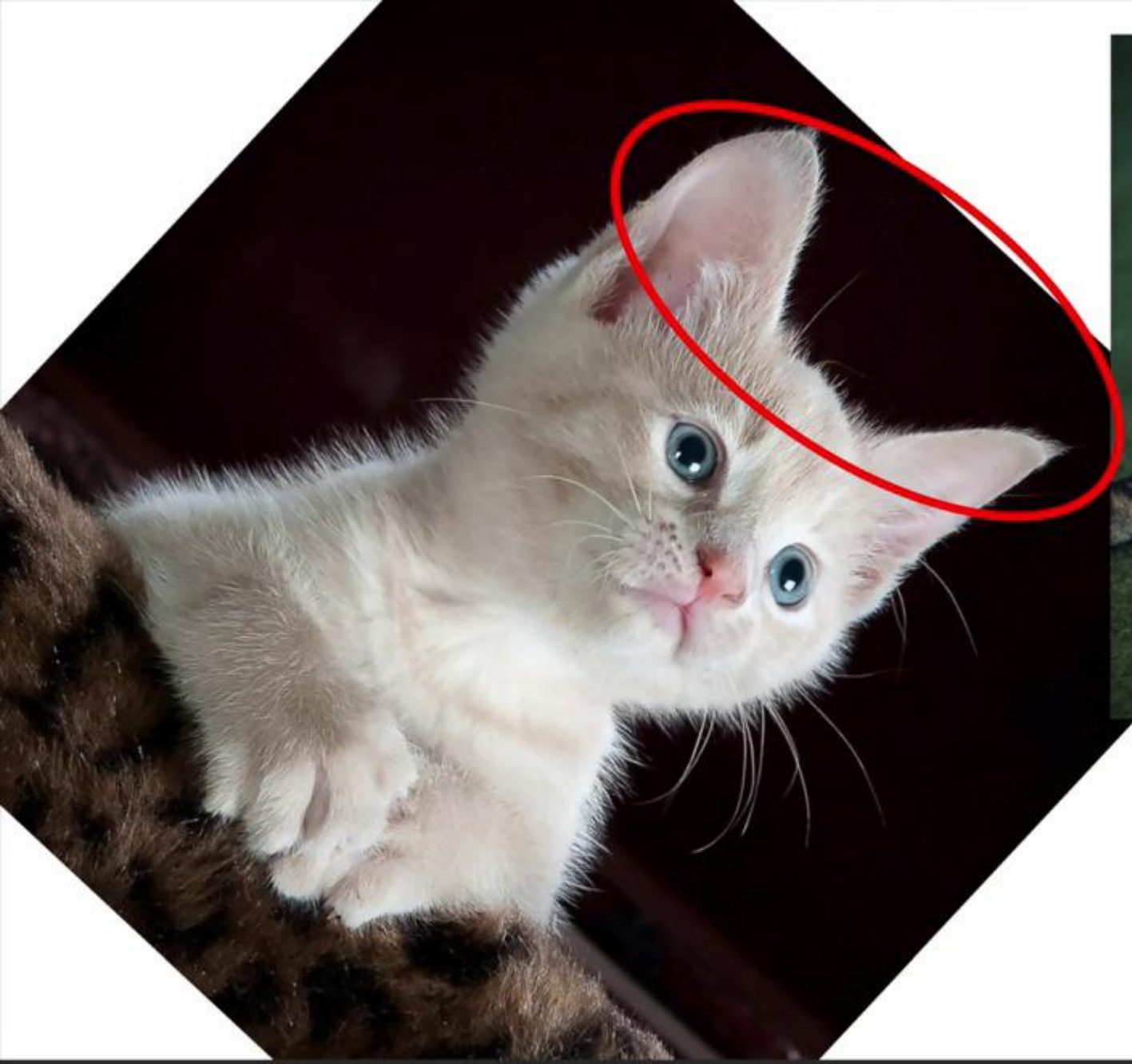
```
test_datagen = ImageDataGenerator(rescale=1./255)

validation_generator = test_datagen.flow_from_directory(
        validation_dir,
        target_size=(150, 150),
        batch_size=20,
        class_mode='binary')
```

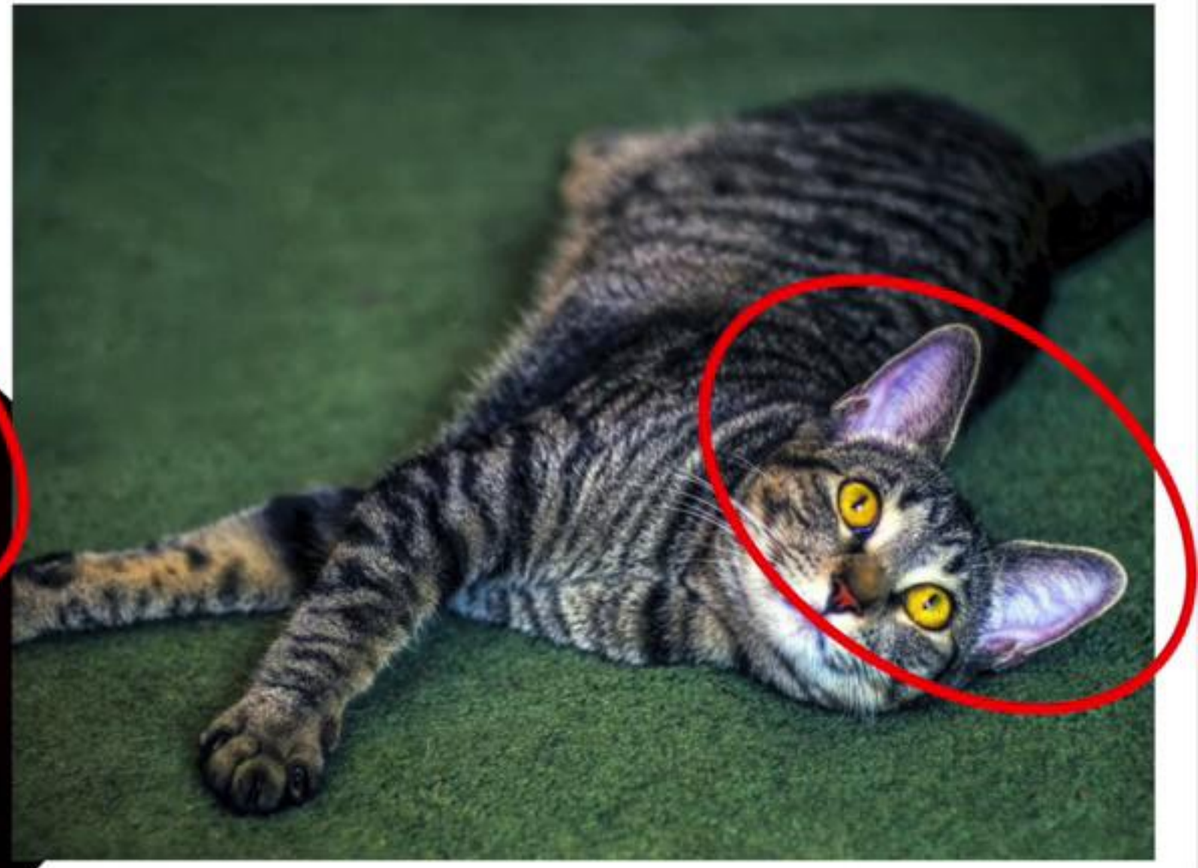
```
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(16, (3,3), activation='relu',
                           input_shape=(150, 150, 3)),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(32, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

```
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 148, 148, 16)      448
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 74, 74, 16)        0
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 72, 72, 32)        4640
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 36, 36, 32)        0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 34, 34, 64)        18496
_________________________________________________________________
max_pooling2d_2 (MaxPooling2 (None, 17, 17, 64)        0
_________________________________________________________________
flatten (Flatten)            (None, 18496)             0
_________________________________________________________________
dense (Dense)                (None, 512)               9470464
_________________________________________________________________
dense_1 (Dense)              (None, 1)                 513
=================================================================
Total params: 9,494,561
Trainable params: 9,494,561
Non-trainable params: 0
```

```
from tensorflow.keras.optimizers import RMSprop


model.compile(loss='binary_crossentropy',
              optimizer=RMSprop(lr=0.001),
              metrics=['acc'])
```

```
history = model.fit_generator(
        train_generator,
        steps_per_epoch=100,
        epochs=15,
        validation_data=validation_generator,
        validation_steps=50,
        verbose=2)
```

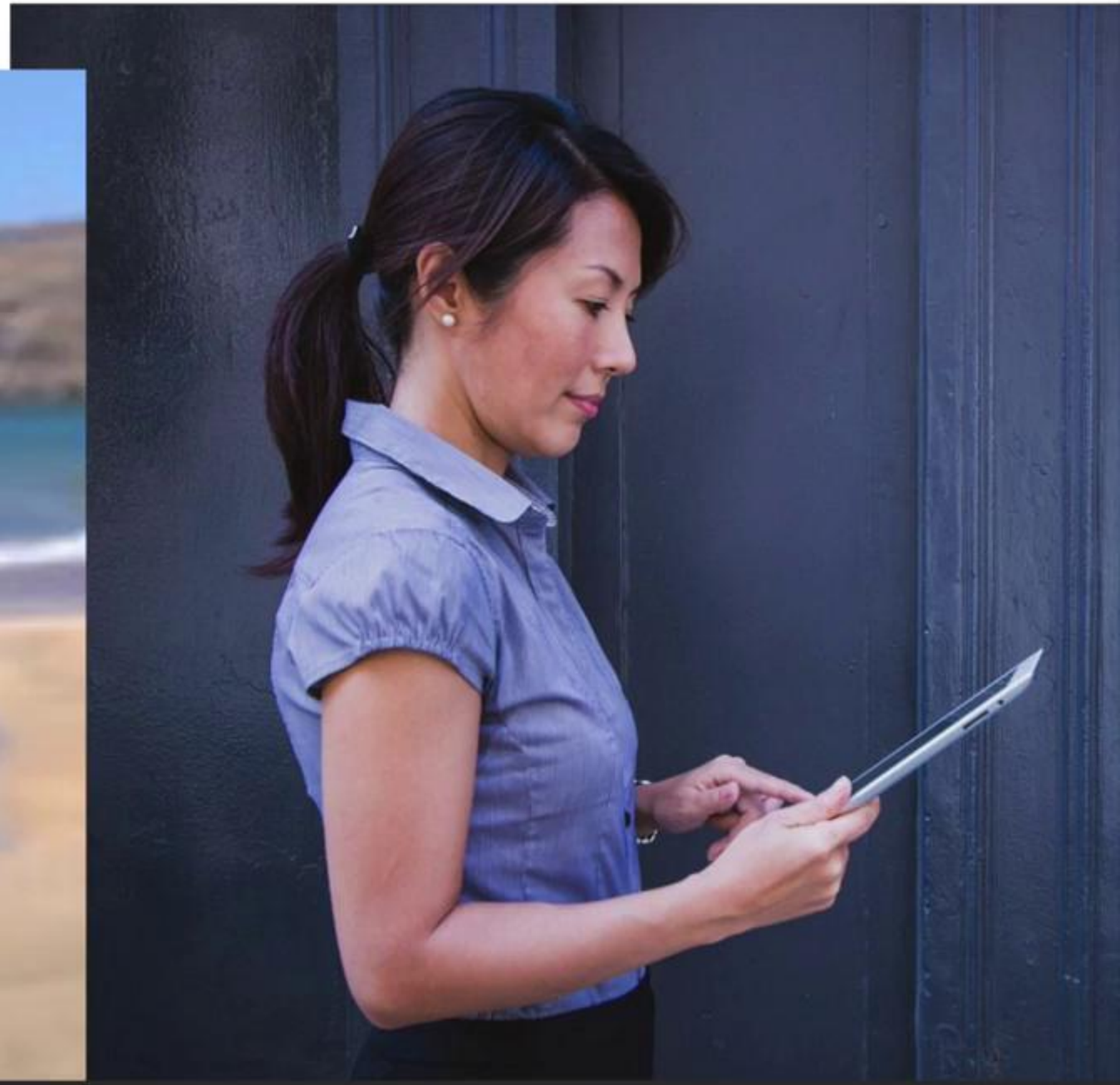




```
train_datagen = ImageDataGenerator(rescale=1./255)
```

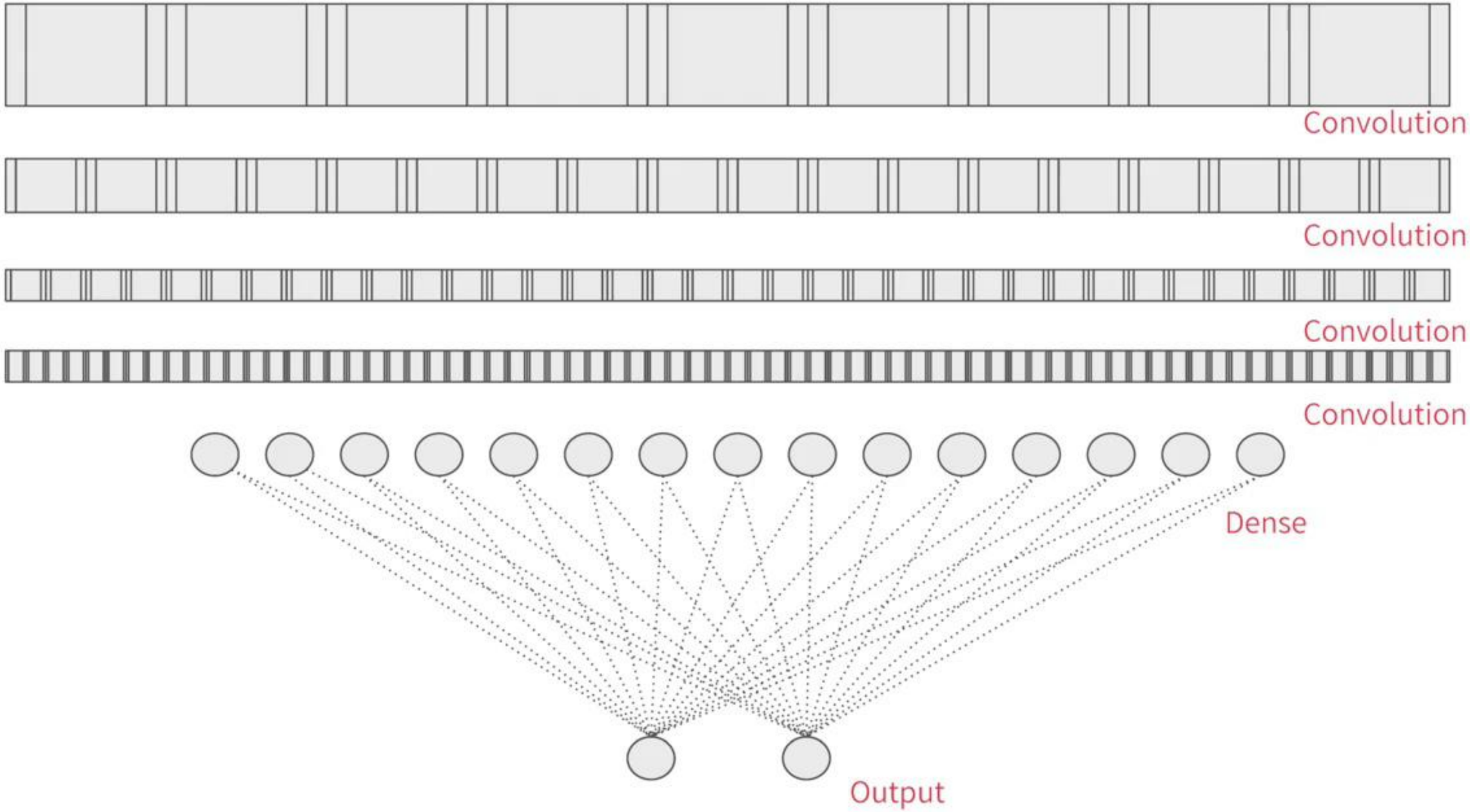
```
# Updated to do image augmentation
train_datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=40,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        fill_mode='nearest')
```

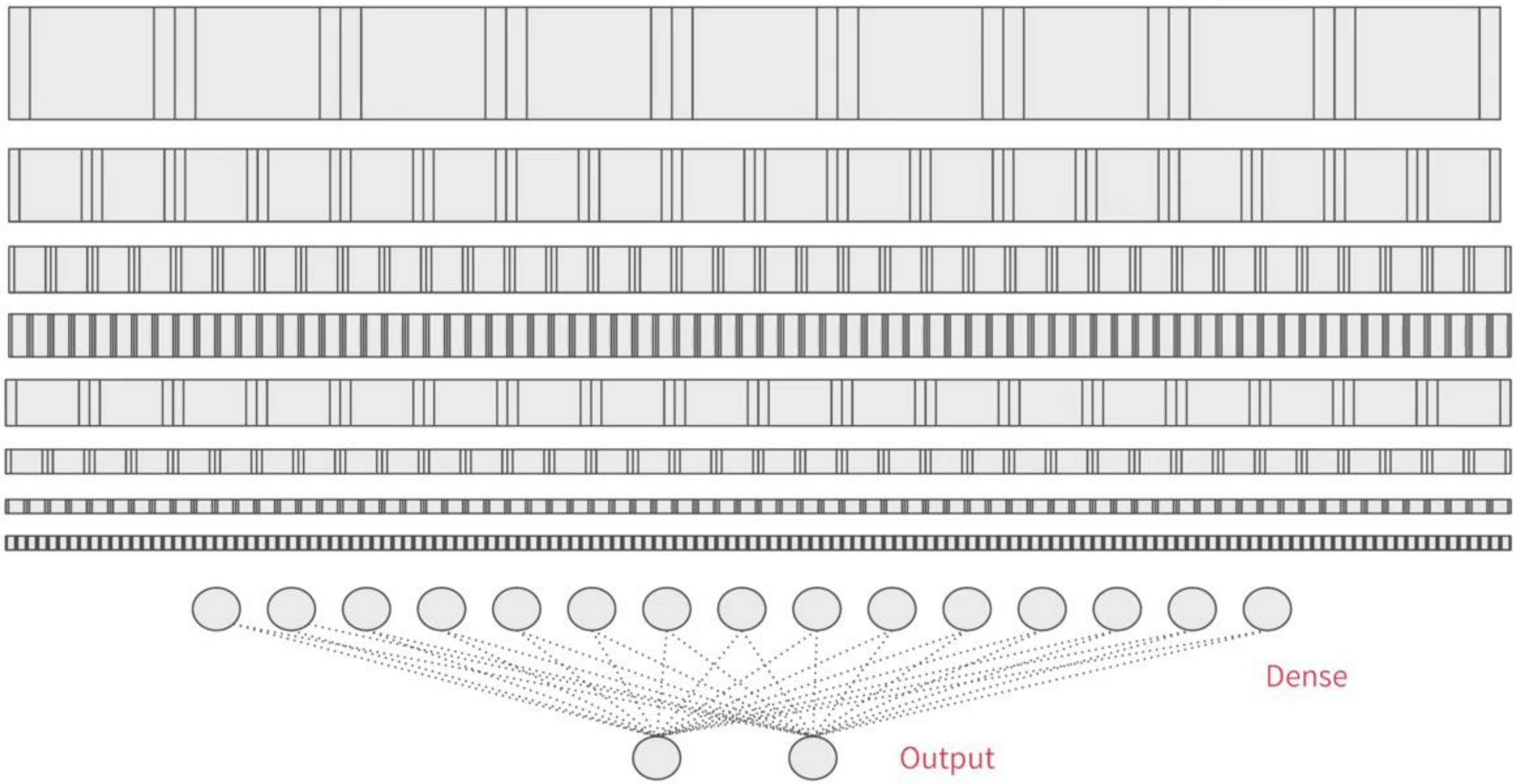
```
# Updated to do image augmentation
train_datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=40,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        fill_mode='nearest')
```

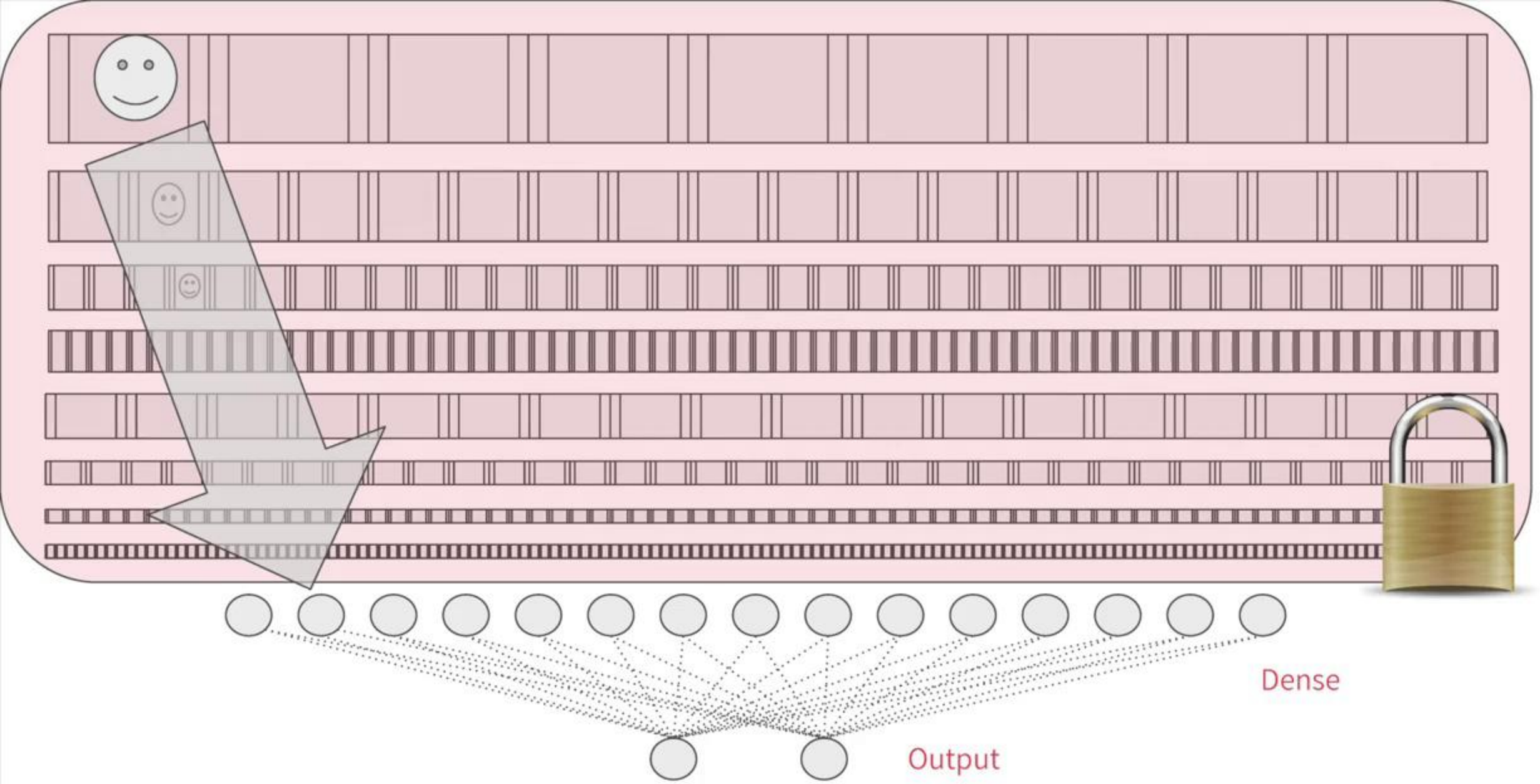


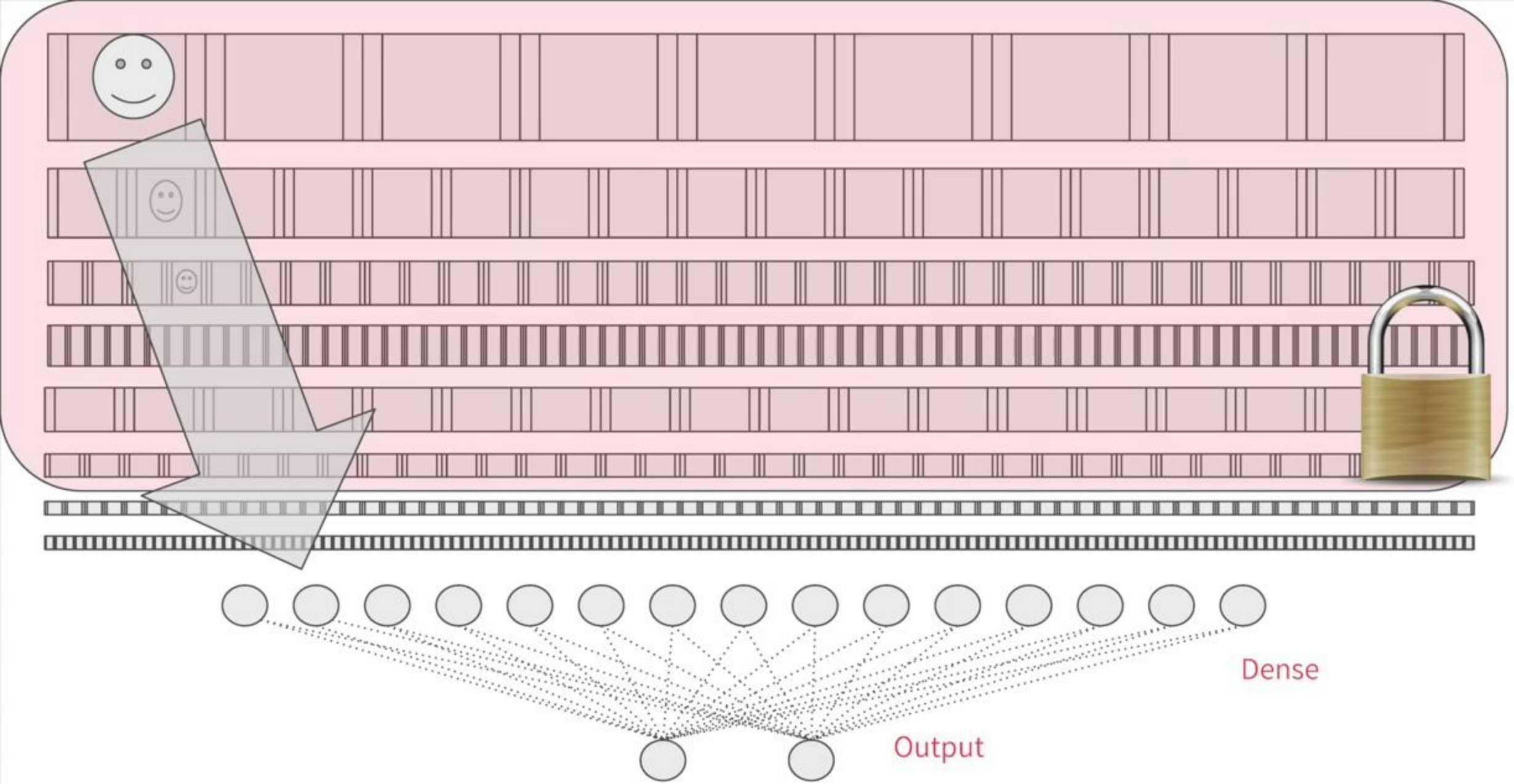
```
# Updated to do image augmentation
train_datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=40,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        fill_mode='nearest')
```

```
# Updated to do image augmentation
train_datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=40,
        width_shift_range=0.2,
        height_shift_range=0.2,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        fill_mode='nearest')
```


Convolution
Convolution
Convolution
Convolution
Dense
Output


Dense
Output


Dense
Output


Dense
Output

https://arxiv.org/abs/1512.00567

Cornell University

We gratefully acknowledge support from the Simons Foundation and member institutions.

arXiv.org > cs > arXiv:1512.00567

Search or Article ID | All fields

(Help | Advanced search)

Computer Science > Computer Vision and Pattern Recognition

Rethinking the Inception Architecture for Computer Vision

Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, Zbigniew Wojna

(Submitted on 2 Dec 2015 (v1), last revised 11 Dec 2015 (this version, v3))

Convolutional networks are at the core of most state-of-the-art computer vision solutions for a wide variety of tasks. Since 2014 very deep convolutional networks started to become mainstream, yielding substantial gains in various benchmarks. Although increased model size and computational cost tend to translate to immediate quality gains for most tasks (as long as enough labeled data is provided for training), computational efficiency and low parameter count are still enabling factors for various use cases such as mobile vision and big-data scenarios. Here we explore ways to scale up networks in ways that aim at utilizing the added computation as efficiently as possible by suitably factorized convolutions and aggressive regularization. We benchmark our methods on the ILSVRC 2012 classification challenge validation set demonstrate substantial gains over the state of the art: 21.2% top-1 and 5.6% top-5 error for single frame evaluation using a network with a computational cost of 5 billion multiply-adds per inference and with using less than 25 million parameters. With an ensemble of 4 models and multi-crop evaluation, we report 3.5% top-5 error on the validation set (3.6% error on the test set) and 17.3% top-1 error on the validation set.

Download:

- PDF
- Other formats

(license)

Current browse context:
cs.CV
< prev | next >
new | recent | 1512

Change to browse by:
cs

References & Citations
- NASA ADS

http://image-net.org/

IMAGENET

14,197,122 images, 21841 synsets indexed

Explore Download Challenges Publications CoolStuff About

Not logged in. Login | Signup

ImageNet is an image database organized according to the WordNet hierarchy (currently only the nouns), in which each node of the hierarchy is depicted by hundreds and thousands of images. Currently we have an average of over five hundred images per node. We hope ImageNet will become a useful resource for researchers, educators, students and all of you who share our passion for pictures.
Click here to learn more about ImageNet, Click here to join the ImageNet mailing list.

What do these images have in common? *Find out!*

Check out the ImageNet Challenge on Kaggle!

© 2016 Stanford Vision Lab, Stanford University, Princeton University support@image-net.org Copyright infringement

```python
import os

from tensorflow.keras import layers
from tensorflow.keras import Model
```

https://storage.googleapis.com/mledu-datasets/inception_v3_weights_tf_dim_ordering_tf_kernels

```python
from tensorflow.keras.applications.inception_v3 import InceptionV3

local_weights_file = '/tmp/inception_v3_weights_tf_dim_ordering_tf_kernels_notop.h5'

pre_trained_model = InceptionV3(input_shape = (150, 150, 3),
                                include_top = False,
                                weights = None)

pre_trained_model.load_weights(local_weights_file)
```

```
for layer in pre_trained_model.layers:
  layer.trainable = False
```

```
pre_trained_model.summary()
```

Model: "inception_v3"

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 299, 299, 3)]	0	
conv2d (Conv2D)	(None, 74, 74, 32)	864	input_1[0][0]
batch_normalization_v1 (BatchNo	(None, 74, 74, 32)	96	conv2d[0][0]
activation (Activation)	(None, 74, 74, 32)	0	batch_normalization_v1[0][0]
conv2d_1 (Conv2D)	(None, 72, 72, 32)	9216	activation[0][0]
batch_normalization_v1_1 (Batch	(None, 72, 72, 32)	96	conv2d_1[0][0]
activation_1 (Activation)	(None, 72, 72, 32)	0	batch_normalization_v1_1[0][0]
conv2d_2 (Conv2D)	(None, 72, 72, 64)	18432	activation_1[0][0]
batch_normalization_v1_2 (Batch	(None, 72, 72, 64)	192	conv2d_2[0][0]
activation_2 (Activation)	(None, 72, 72, 64)	0	batch_normalization_v1_2[0][0]
max_pooling2d (MaxPooling2D)	(None, 35, 35, 64)	0	activation_2[0][0]
conv2d_3 (Conv2D)	(None, 35, 35, 80)	5120	max_pooling2d[0][0]
batch_normalization_v1_3 (Batch	(None, 35, 35, 80)	240	conv2d_3[0][0]
activation_3 (Activation)	(None, 35, 35, 80)	0	batch_normalization_v1_3[0][0]
conv2d_4 (Conv2D)	(None, 33, 33, 192)	138240	activation_3[0][0]
batch_normalization_v1_4 (Batch	(None, 33, 33, 192)	576	conv2d_4[0][0]
activation_4 (Activation)	(None, 33, 33, 192)	0	batch_normalization_v1_4[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 16, 16, 192)	0	activation_4[0][0]
conv2d_8 (Conv2D)	(None, 16, 16, 64)	12288	max_pooling2d_1[0][0]
batch_normalization_v1_8 (Batch	(None, 16, 16, 64)	192	conv2d_8[0][0]
activation_8 (Activation)	(None, 16, 16, 64)	0	batch_normalization_v1_8[0][0]
conv2d_6 (Conv2D)	(None, 16, 16, 48)	9216	max_pooling2d_1[0][0]
conv2d_9 (Conv2D)	(None, 16, 16, 96)	55296	activation_8[0][0]
batch_normalization_v1_6 (Batch	(None, 16, 16, 48)	144	conv2d_6[0][0]
batch_normalization_v1_9 (Batch	(None, 16, 16, 96)	288	conv2d_9[0][0]
activation_6 (Activation)	(None, 16, 16, 48)	0	batch_normalization_v1_6[0][0]
activation_9 (Activation)	(None, 16, 16, 96)	0	batch_normalization_v1_9[0][0]
average_pooling2d (AveragePooli	(None, 16, 16, 192)	0	max_pooling2d_1[0][0]
conv2d_5 (Conv2D)	(None, 16, 16, 64)	12288	max_pooling2d_1[0][0]
conv2d_7 (Conv2D)	(None, 16, 16, 64)	76800	activation_6[0][0]
conv2d_10 (Conv2D)	(None, 16, 16, 96)	82944	activation_9[0][0]
conv2d_11 (Conv2D)	(None, 16, 16, 32)	6144	average_pooling2d[0][0]
batch_normalization_v1_5 (Batch	(None, 16, 16, 64)	192	conv2d_5[0][0]
batch_normalization_v1_7 (Batch	(None, 16, 16, 64)	192	conv2d_7[0][0]
batch_normalization_v1_10 (Batc	(None, 16, 16, 96)	288	conv2d_10[0][0]
batch_normalization_v1_11 (Batc	(None, 16, 16, 32)	96	conv2d_11[0][0]
activation_5 (Activation)	(None, 16, 16, 64)	0	batch_normalization_v1_5[0][0]
activation_7 (Activation)	(None, 16, 16, 64)	0	batch_normalization_v1_7[0][0]
activation_10 (Activation)	(None, 16, 16, 96)	0	batch_normalization_v1_10[0][0]
activation_11 (Activation)	(None, 16, 16, 32)	0	batch_normalization_v1_11[0][0]
mixed0 (Concatenate)	(None, 16, 16, 256)	0	activation_5[0][0] activation_7[0][0] activation_10[0][0] activation_11[0][0]
conv2d_15 (Conv2D)	(None, 16, 16, 64)	16384	mixed0[0][0]

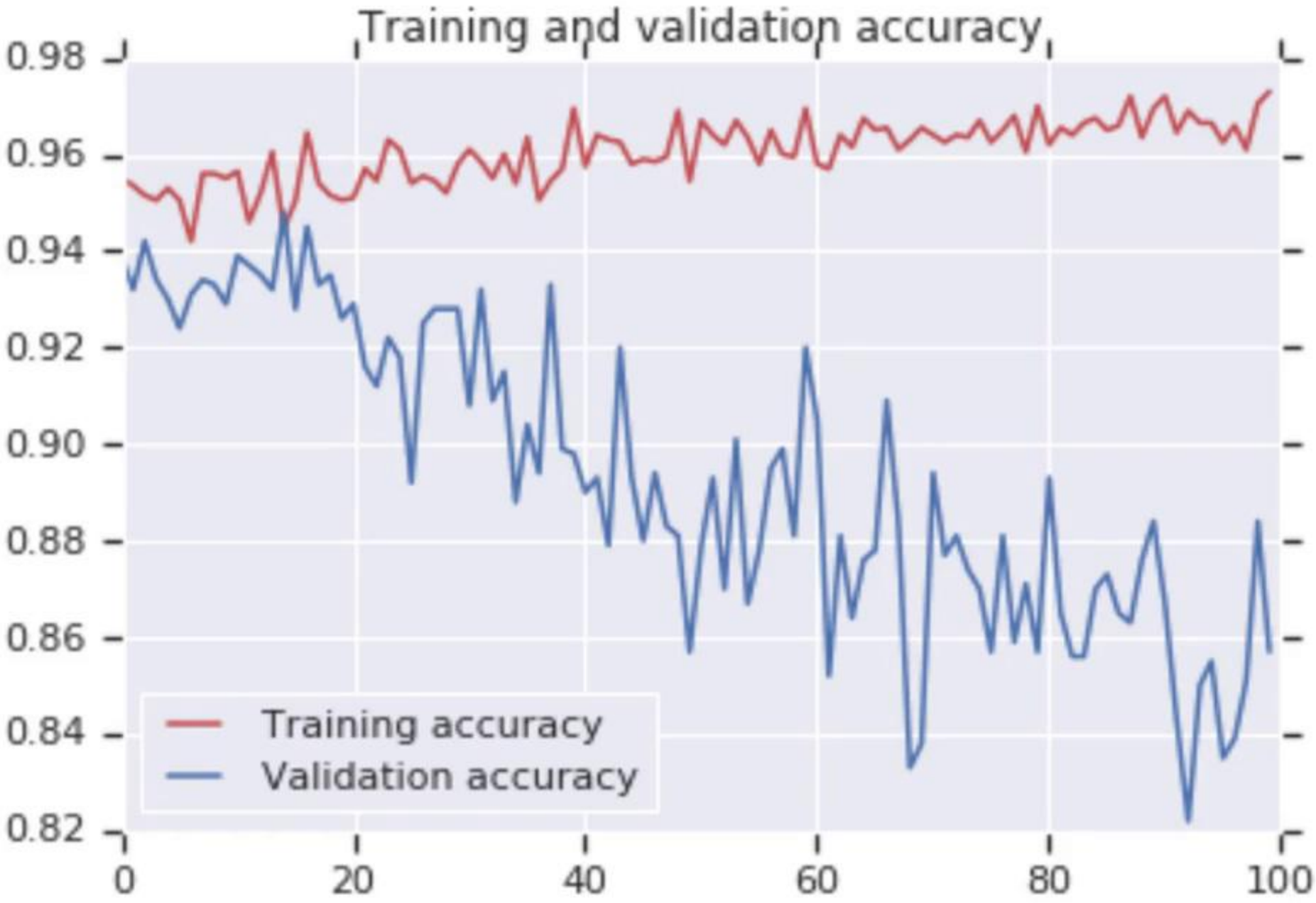
```
last_layer = pre_trained_model.get_layer('mixed7')

last_output = last_layer.output
```

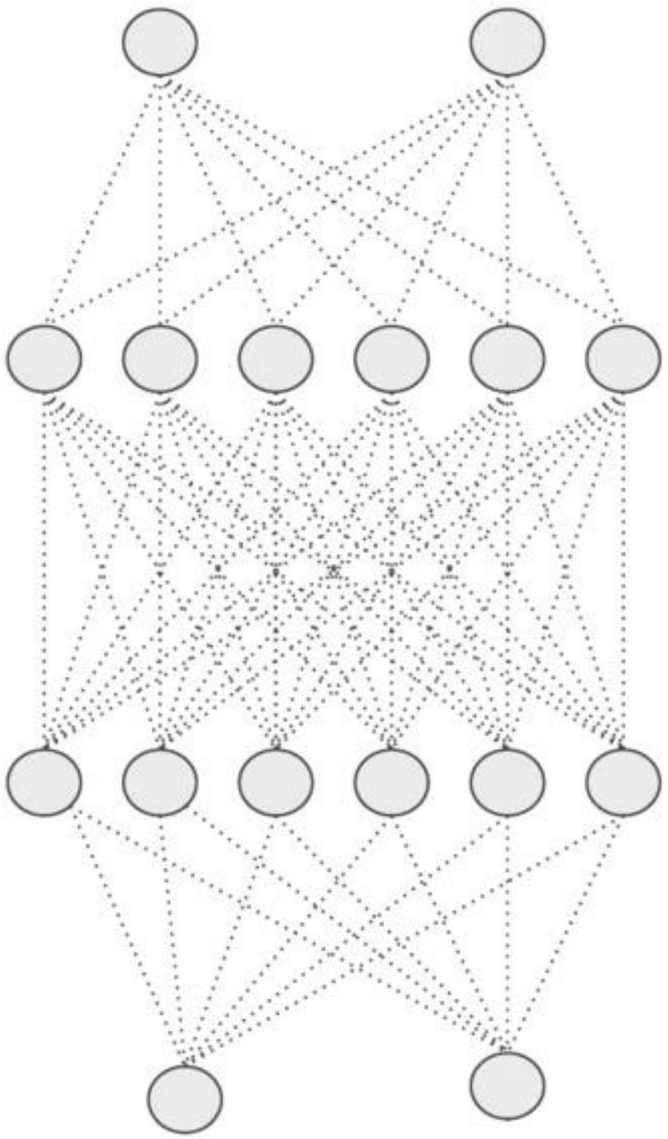
```
from tensorflow.keras.optimizers import RMSprop

x = layers.Flatten()(last_output)
x = layers.Dense(1024, activation='relu')(x)
x = layers.Dense  (1, activation='sigmoid')(x)

model = Model( pre_trained_model.input, x)
model.compile(optimizer = RMSprop(learning_rate=0.0001),
              loss = 'binary_crossentropy',
              metrics = ['acc'])
```

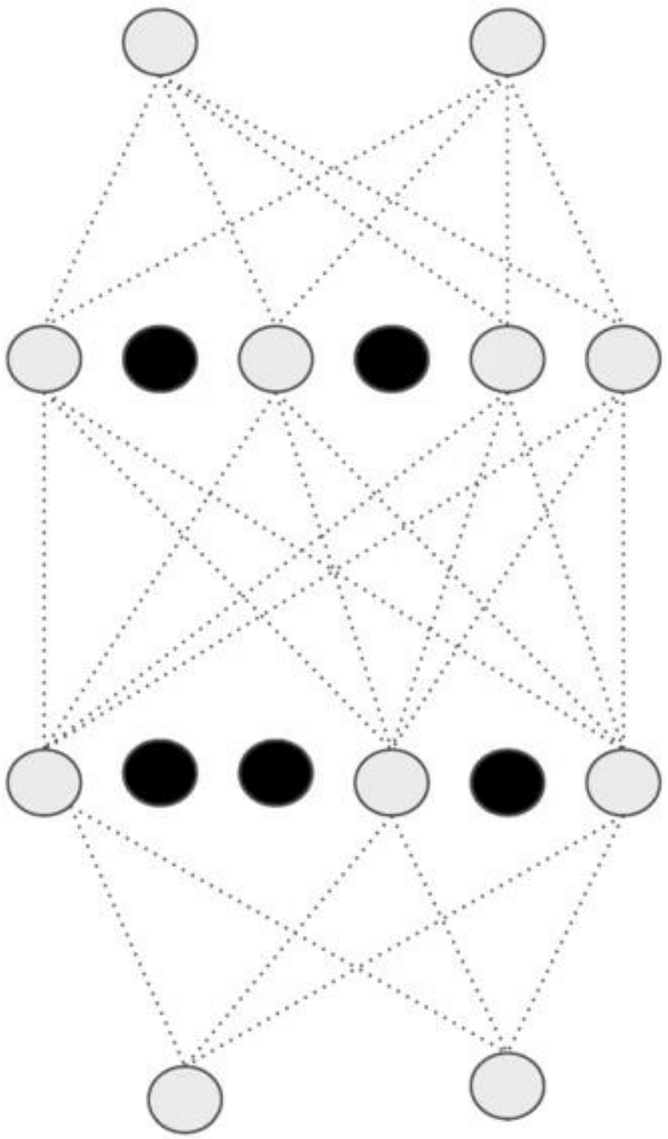
```
# Add our data-augmentation parameters to ImageDataGenerator
train_datagen = ImageDataGenerator(rescale = 1./255.,
                                   rotation_range = 40,
                                   width_shift_range = 0.2,
                                   height_shift_range = 0.2,
                                   shear_range = 0.2,
                                   zoom_range = 0.2,
                                   horizontal_flip = True)
```

```
train_generator = train_datagen.flow_from_directory(
                    train_dir,
                    batch_size = 20,
                    class_mode = 'binary',
                    target_size = (150, 150))
```

```
history = model.fit(
            train_generator,
            validation_data = validation_generator,
            steps_per_epoch = 100,
            epochs = 100,
            validation_steps = 50,
            verbose = 2)
```

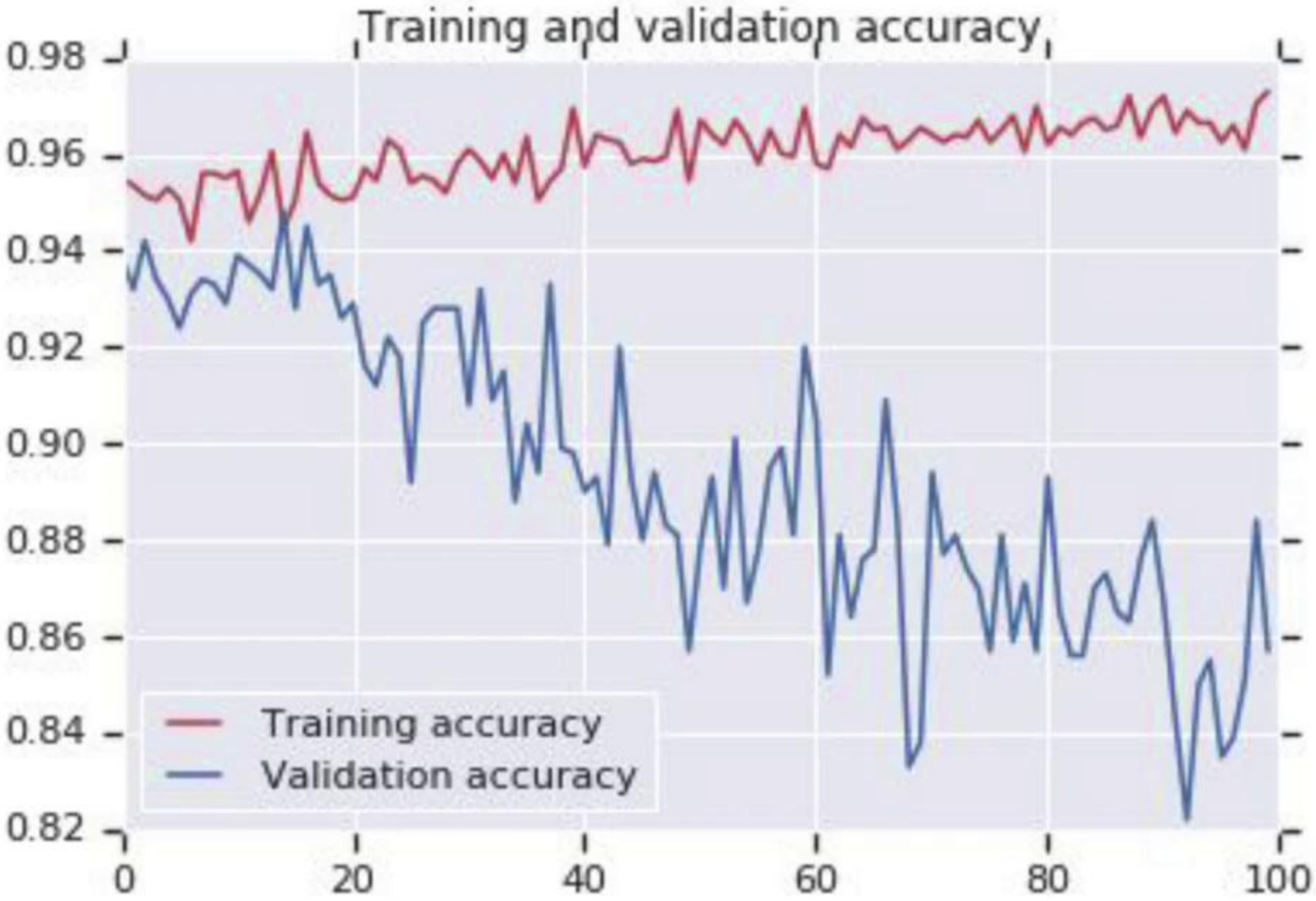

Training and validation accuracy
0.98
0.96
0.94
0.92
0.90
0.88
0.86
0.84
0.82
0
20
40
60
80
100
Training accuracy
Validation accuracy

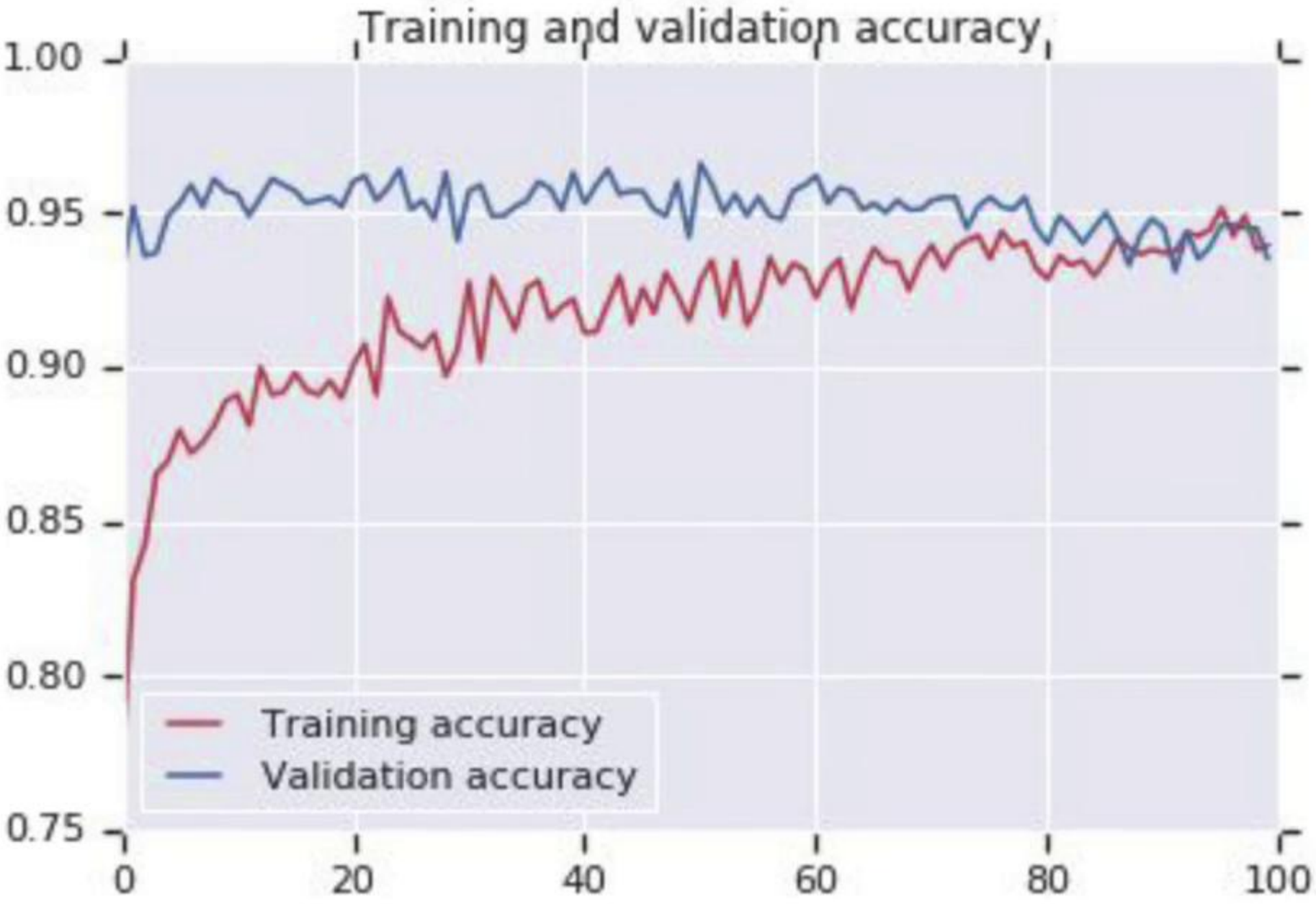


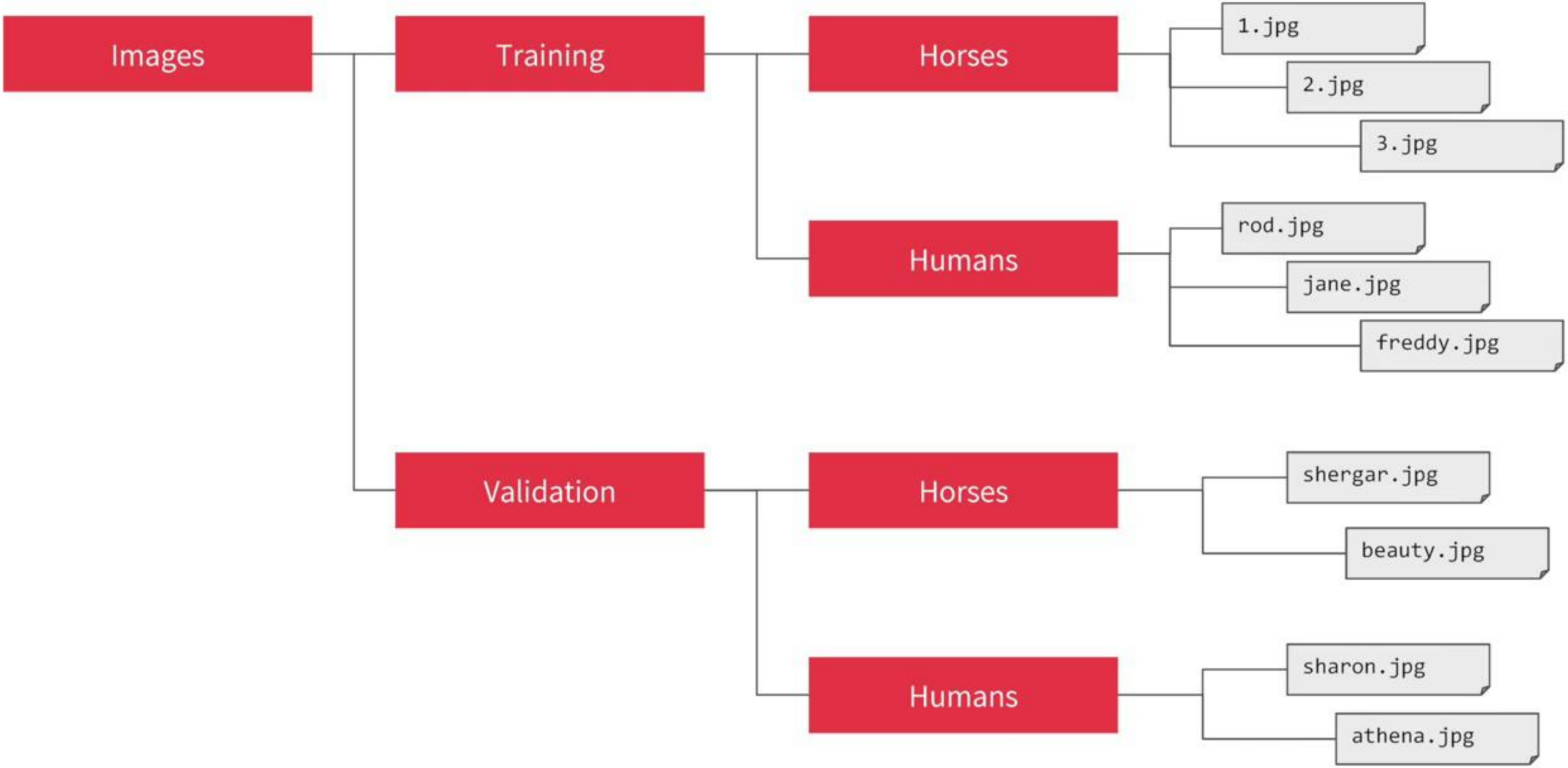


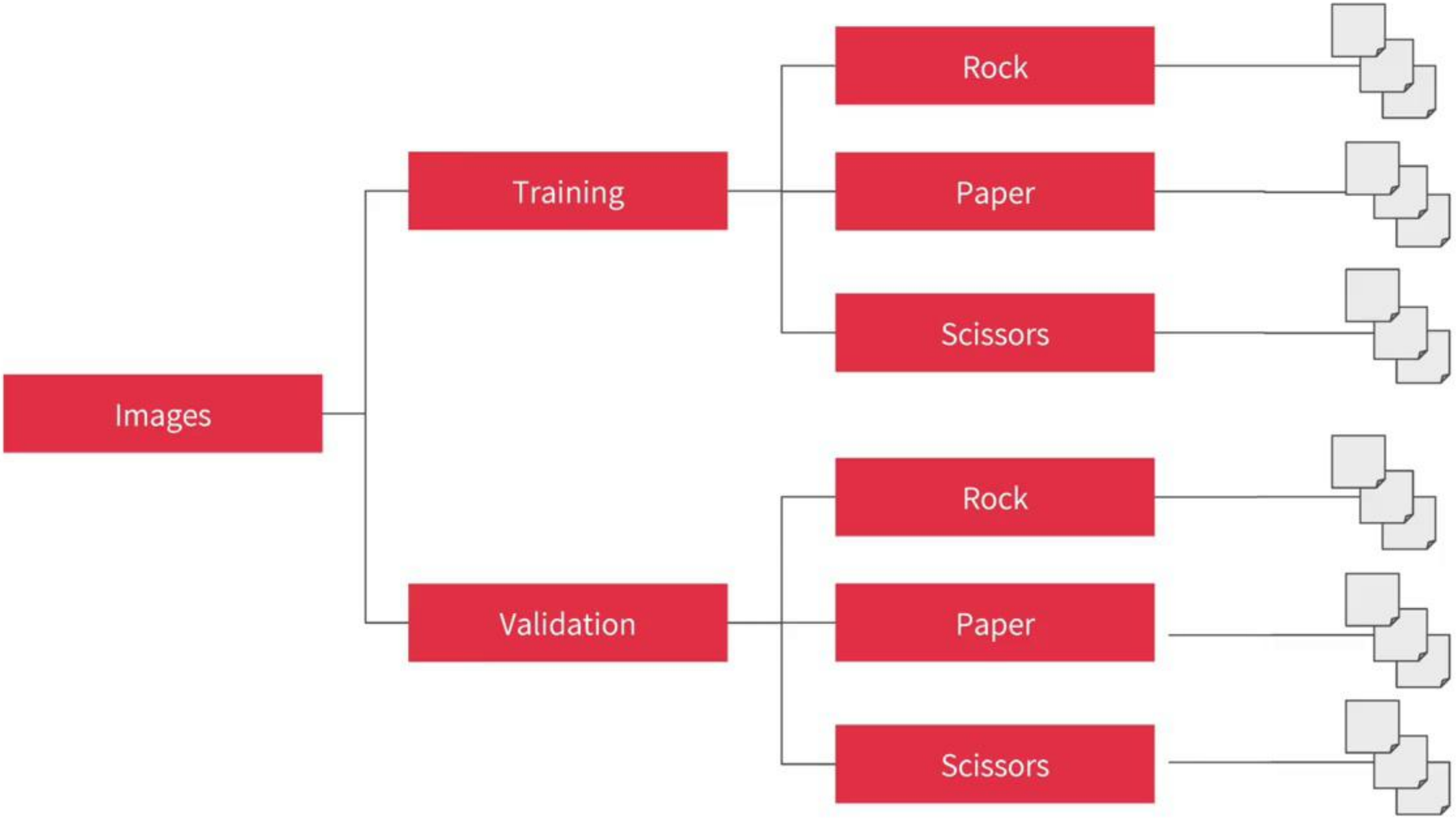
```
from tensorflow.keras.optimizers import RMSprop

x = layers.Flatten()(last_output)
x = layers.Dense(1024, activation='relu')(x)
x = layers.Dense  (1, activation='sigmoid')(x)

model = Model( pre_trained_model.input, x)
model.compile(optimizer = RMSprop(lr=0.0001),
              loss = 'binary_crossentropy',
              metrics = ['acc'])
```

```
from tensorflow.keras.optimizers import RMSprop

x = layers.Flatten()(last_output)
x = layers.Dense(1024, activation='relu')(x)
x = layers.Dropout(0.2)(x)
x = layers.Dense  (1, activation='sigmoid')(x)

model = Model( pre_trained_model.input, x)
model.compile(optimizer = RMSprop(lr=0.0001),
              loss = 'binary_crossentropy',
              metrics = ['acc'])
```


Training and validation accuracy
0.98
0.96
0.94
0.92
0.90
0.88
0.86
0.84
0.82
0
20
40
60
80
100
Training accuracy
Validation accuracy


Training and validation accuracy
1.00
0.95
0.90
0.85
0.80
0.75
0
20
40
60
80
100
Training accuracy
Validation accuracy


Images
Training
Horses
1.jpg
2.jpg
3.jpg
Humans
rod.jpg
jane.jpg
freddy.jpg
Validation
Horses
shergar.jpg
beauty.jpg
Humans
sharon.jpg
athena.jpg


Images
Training
Validation
Rock
Paper
Scissors
Rock
Paper
Scissors

http://www.laurencemoroney.com/rock-paper-scissors-dataset/

Rock Paper Scissors Dataset

Introducing Rock Paper Scissors – A multi class learning dataset

Abstract

Rock Paper Scissors is a dataset containing 2,892 images of diverse hands in Rock/Paper/Scissors poses. It is licensed CC By 2.0 and available for all purposes, but it's intent is primarily for learning and research.

Overview

Rock Paper Scissors contains images from a variety of different hands, from different races, ages and genders, posed into Rock / Paper or Scissors and labelled as such. You can download the training set here, and the test set here. These images have all been generated using CGI techniques as an experiment in determining if a CGI-based dataset can be

FOLLOW ME ON TWITTER

Laurence Moroney Retweeted

TensorFlow
@TensorFlow

In the third episode of the Intro to Google Colaboratory series, @lmoroney covers how to quickly build a neural network for basic Breast Cancer classification.

Watch this #CodingTensorFlow → bit.ly/2SPD4XC

P.S. Don't forget to add your homework below!

```
train_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
        train_dir,
        target_size=(300, 300),
        batch_size=128,
        class_mode='binary')
```

```
model = tf.keras.models.Sequential([
    tf.keras.layers.Conv2D(16, (3,3), activation='relu',
                           input_shape=(300, 300, 3)),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(32, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

Rock: 0.001 Paper: 0.647 Scissors: 0.352

```
from tensorflow.keras.optimizers import RMSprop

model.compile(loss='binary_crossentropy',
              optimizer=RMSprop(lr=0.001),
              metrics=['acc'])
```

```
from tensorflow.keras.optimizers import RMSprop

model.compile(loss='categorical_crossentropy',
              optimizer=RMSprop(lr=0.001),
              metrics=['acc'])
```

Training and validation accuracy
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0
20
40
60
80
100
Training accuracy
Validation accuracy